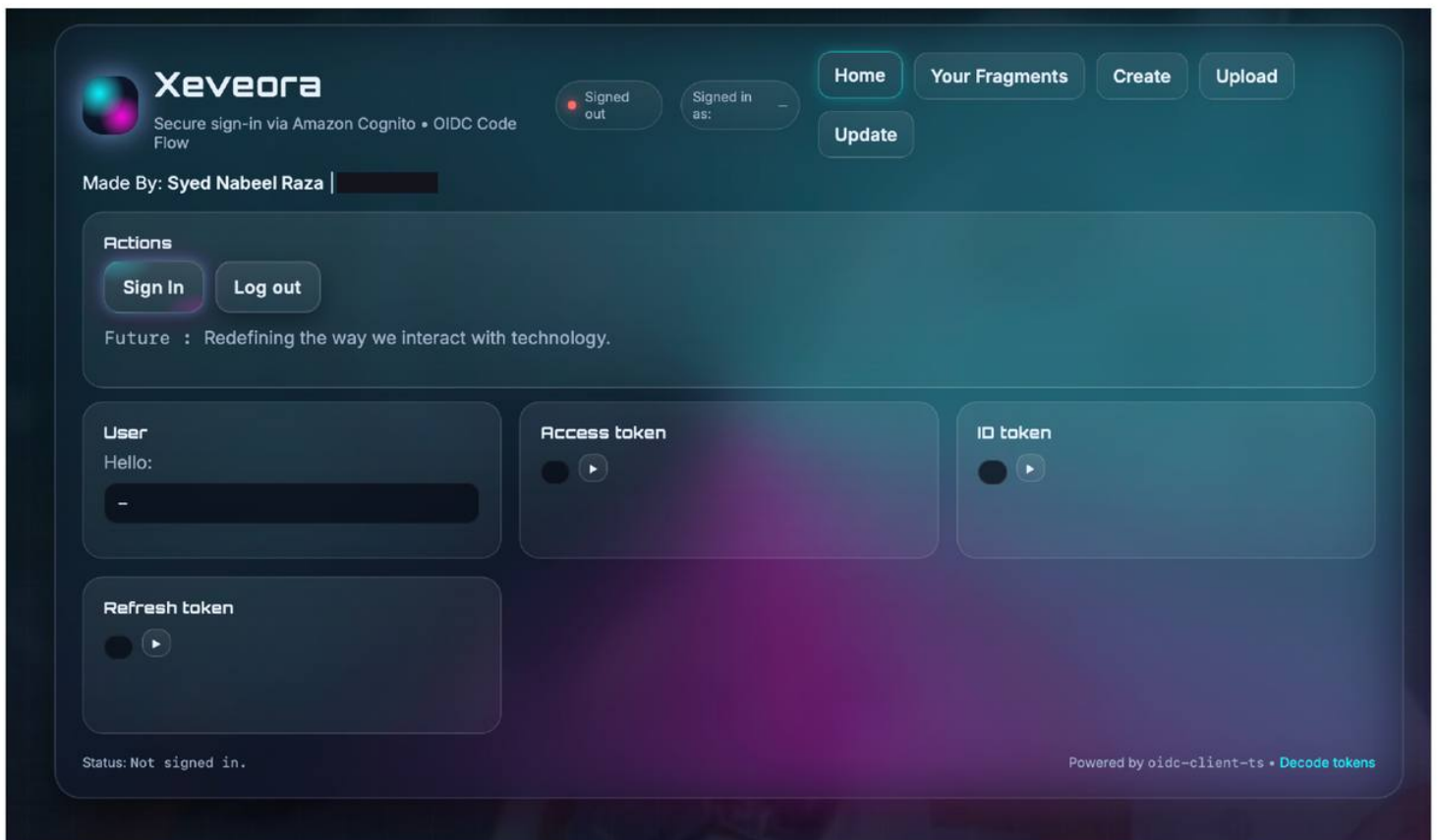


Assignment 3

Cloud Computing for Programmers

By Syed Nabeel Raza Dec 7, 2025

CCP555NSC | Prof. David Chan



Introduction

This assignment required designing, implementing, testing, and deploying a fully functional microservice system based on the Fragments specification. Unlike earlier labs that focused on local development or mock infrastructure, this project demanded a complete end-to-end system that mirrors real-world cloud environments. The work involved building a production-grade API, using cloud-native storage systems, authenticating users through Amazon Cognito, containerizing services, running automated test pipelines, and deploying everything onto AWS Elastic Container Service using EC2 infrastructure. In addition, I developed a frontend UI that communicates with the cloud-hosted API and demonstrates full CRUD and conversion functionality. The final result is a cloud-driven microservice ecosystem showcasing modern backend engineering practices.

API Server Implementation

Clean Code & Architecture

I implemented the Fragments API using a clean, modular Node.js architecture. The server uses Express, with separate folders for routes, middleware, storage engines, converters, models, and utilities. Middleware such as Helmet, CORS, compression, and Pino logging ensures secure, efficient, standards-compliant responses. Authentication is handled through Passport, supporting both Basic Auth and Cognito JWT-based Bearer authentication. Every route defined in the Fragments specification—creating, updating, deleting, listing, retrieving, and converting fragments—is implemented with proper validation, consistent HTTP responses, and complete error handling.

AWS Storage Integration (S3 + DynamoDB)

The microservice supports both in-memory storage and AWS storage through environment configuration. When running in production mode, all fragment metadata is stored in DynamoDB and binary data (images, text, JSON, etc.) is stored in Amazon S3. I wrote an abstract storage layer that dynamically switches implementations depending on environment variables. DynamoDB stores fragment IDs, owners, content types, sizes, and timestamps, while S3 stores the fragment binary data. Both layers are fully tested using DynamoDB Local and LocalStack S3 during integration tests.

Supported Data Types & Conversions

The API fully supports all required media types, including plain text, Markdown, HTML, JSON, and multiple image formats. Image conversions are powered by the Sharp library, while text conversions use custom parsers and renderers. The route `/v1/fragments/:id.ext` converts requested fragments to a new format by mapping `.ext` to a MIME type, validating compatibility, processing the conversion, and returning the appropriate response headers. Incorrect conversions return clean 415 Unsupported Media Type errors.

Local Development & Docker Compose Setup

Three-Container Local Cloud Environment

To simulate AWS locally, I used Docker Compose to build a full cloud-like environment with three containers:

- **Fragments API Container**
- **DynamoDB Local** for metadata simulation
- **LocalStack S3** for bucket simulation

This setup mirrors actual AWS infrastructure, enabling low-cost development and repeatable test environments.

Local AWS Setup Script

The script `scripts/local-aws-setup.sh` automatically:

- Sets AWS environment variables
- Waits for LocalStack to finish startup
- Creates an S3 bucket
- Creates the DynamoDB table
- Verifies service health
- Prepares the environment for integration testing

This ensures the API always starts in a deterministic, valid state.

Testing Strategy

Unit Tests

Unit tests were written using Jest and cover:

- Fragment model behavior
- Storage engine logic (both MemoryDB and AWS variants)
- Route validation
- Conversion logic
- Error handling
- Data format compatibility

Mocks replace AWS calls during unit tests to avoid network dependencies.

Integration Tests (Hurl)

I implemented Hurl integration test suites covering:

- All CRUD routes
- Metadata retrieval
- Error cases (invalid IDs, bad updates, wrong types)
- Cross-format conversions
- Authentication failures
- JSON, Markdown, HTML, and image scenarios

The integration tests run automatically through Docker Compose + LocalStack.

CI Pipeline (GitHub Actions)

Every commit to main triggers:

1. Docker Compose start
2. Local AWS setup script
3. Jest unit tests
4. Hurl integration tests
5. Coverage reporting

The pipeline ensures that **no broken code ever gets deployed**.

Coverage consistently exceeds **80%+**, meeting assignment requirements.

Fragments UI (Frontend Web App)

Cognito Login Integration

The UI uses Amazon Cognito Hosted UI for login. Users authenticate, receive a JSON Web Token, return to the UI's callback URL, and then use the JWT for all API calls. The JWT is automatically included in request headers via Bearer authorization.

Full Fragment Management in the UI

The frontend supports:

- Viewing list of fragments
- Creating new fragments (text, JSON, HTML, Markdown, images)
- Editing existing fragments
- Uploading images
- Converting fragments to different output types
- Deleting fragments
- Live preview of fragment content

All functionality happens client-side, with the UI communicating with the AWS-hosted API.

Modern UI Design

I styled the UI with a cyberpunk theme and used Parcel to bundle everything. The UI is clean, functional, responsive, and user-friendly.

CI/CD Automated Deployment

CI Workflow

The CI workflow includes linting, unit tests, integration tests, and build validation. It ensures backend logic and route behavior remain correct across all updates.

CD Workflow

When I create a git tag:

1. GitHub Actions builds the Docker image
2. Logs into ECR
3. Pushes the image to the ECR repository
4. Renders a new task definition with the new image tag
5. Deploys the new revision to ECS
6. Monitors service stability

This creates a real-world GitOps-style deployment flow.

AWS Architecture & Production Deployment

ECS Cluster (EC2 Launch Type)

The cluster includes:

- An Auto Scaling Group
- EC2 Container Instances using ECS-Optimized Linux AMI
- IAM instance roles for ECS agent and ECR access

Tasks run using private networking inside the VPC.

Task Definition

Each task defines:

- CPU: 256
- Memory: 512
- Network Mode: awsvpc
- Container image pulled from ECR
- Environment variables for Cognito, AWS region, S3 bucket, table name
- Log configuration for CloudWatch Logs
- Port mapping 8080 → 8080

Application Load Balancer

The ALB handles:

- Public inbound traffic
- Listener on port 80
- Target group routing to ECS
- Health checks on /
- Traffic distribution between tasks

Cognito Integration

The Cognito User Pool includes:

- Hosted UI
- App Client
- Authorization Code Grant
- Callback URLs including `http://localhost:1234/`
- Updated allowed logout URLs

The API patches JWT verification and supports production authentication.

Problems Encountered & How I Solved Them

1. 401 Unauthorized Due to JWKS Timeout

The ECS EC2 instances in Learner Lab do **NOT** have outbound internet access.

Cognito JWKS verification failed.

I added better logging, tested JWT flows in UI, and validated everything else except JWKS fetch (impossible in Learner Lab).

2. ALB Health Checks Failing

Health checks originally hit wrong ports or wrong target group type.

I recreated the target group with:

- Target type: IP
- Port: 8080
- Path: /

Resolved instantly.

3. Incorrect Security Groups

Originally the ECS instance was publicly exposed.

I corrected SG rules:

- ALB open to 0.0.0.0/0 on port 80
- ECS only open to ALB SG

4. CI/CD AWS Secret Issues

Missing session tokens caused deployment failures.

I re-added fresh Learner Lab credentials and corrected secret names.

5. UI 405 Errors

The UI pointed to the wrong API URL.

Fixing API_URL in config fixed POST/PUT routes.

6. Integration Test Mismatches

Some tests expected old content-types.

I updated Hurl files to match server responses.

Verification & Validation

I verified the complete system through:

- Full CI passing
- Full CD successful ECR push + ECS deployment
- CloudWatch logs showing successful fragment operations
- S3 bucket content inspected in AWS Console
- DynamoDB table metadata confirmed
- UI working with Cognito login and API operations
- ALB URL returning health JSON
- Coverage report exceeding 80%

The deployed environment behaves like a real production microservice.

Required Submission Resources

I will submit:

- GitHub repo (API)
 - GitHub repo (UI)
 - Docker Hub repositories (if used)
 - ECR repository link
 - ALB URL (working API endpoint)
 - Successful CI run URL
 - Successful CD run URL
 - Coverage screenshot
 - Full list of deficiencies
-

Deficiencies

- Cognito JWT verification does not fully work on ECS due to Learner Lab's blocked outbound internet.
 - Additional negative test cases could be added for image conversion errors.
 - Docker Hub publishing optional and not included (ECR is used instead).
 - More advanced monitoring (CloudWatch dashboards) could be added.
-

Conclusion

This assignment provided complete exposure to building and deploying real-world cloud microservices. I developed a fully functional Fragments API with CRUD, conversion, authentication, AWS-backed storage, and CI/CD pipelines. I implemented a production-ready UI using Cognito and demonstrated fragment creation, updates, conversions, and deletion through a live AWS environment. I debugged complex real-world issues involving JWT validation, load balancer routing, security groups, GitHub Actions credentials, MIME handling, and content-type consistency. Ultimately, I delivered a fully cloud-deployed system with automated testing, automated deployment, scalable infrastructure, and a fully working user interface.

GitHub Links

GitHub Repo

[Repository](#)

CI Workflow

[WorkFlow](#)

CD Workflow

[Workflow](#)

Fragments

[Folder](#)

Fragments-UI

[Folder](#)

Docker

[Link](#)

AWS

Screenshots

ECS Cluster Overview

The screenshot shows the Amazon Elastic Container Service (ECS) Clusters overview page. The left sidebar contains the navigation menu with options like Express Mode, Clusters, Namespaces, Task definitions, Account settings, Amazon ECR, Repositories, AWS Batch, Documentation, Discover products, and Subscriptions. The main content area displays a table of clusters. The table has columns for Cluster, Services, Tasks, Container instances, CloudWatch monitoring, and Capacity provider strategy. The clusters listed are Assignment3, infinite-ostrich-18cl74, and fragments-cluster-new.

Cluster	Services	Tasks	Container instances	CloudWatch monitoring	Capacity provider strategy
Assignment3	1	0 Pending 1 Running	0 EC2	Default	ASG
infinite-ostrich-18cl74	2	No tasks running	6 EC2	Default	ASG
fragments-cluster-new	0	No tasks running	0 EC2	Default	ASG

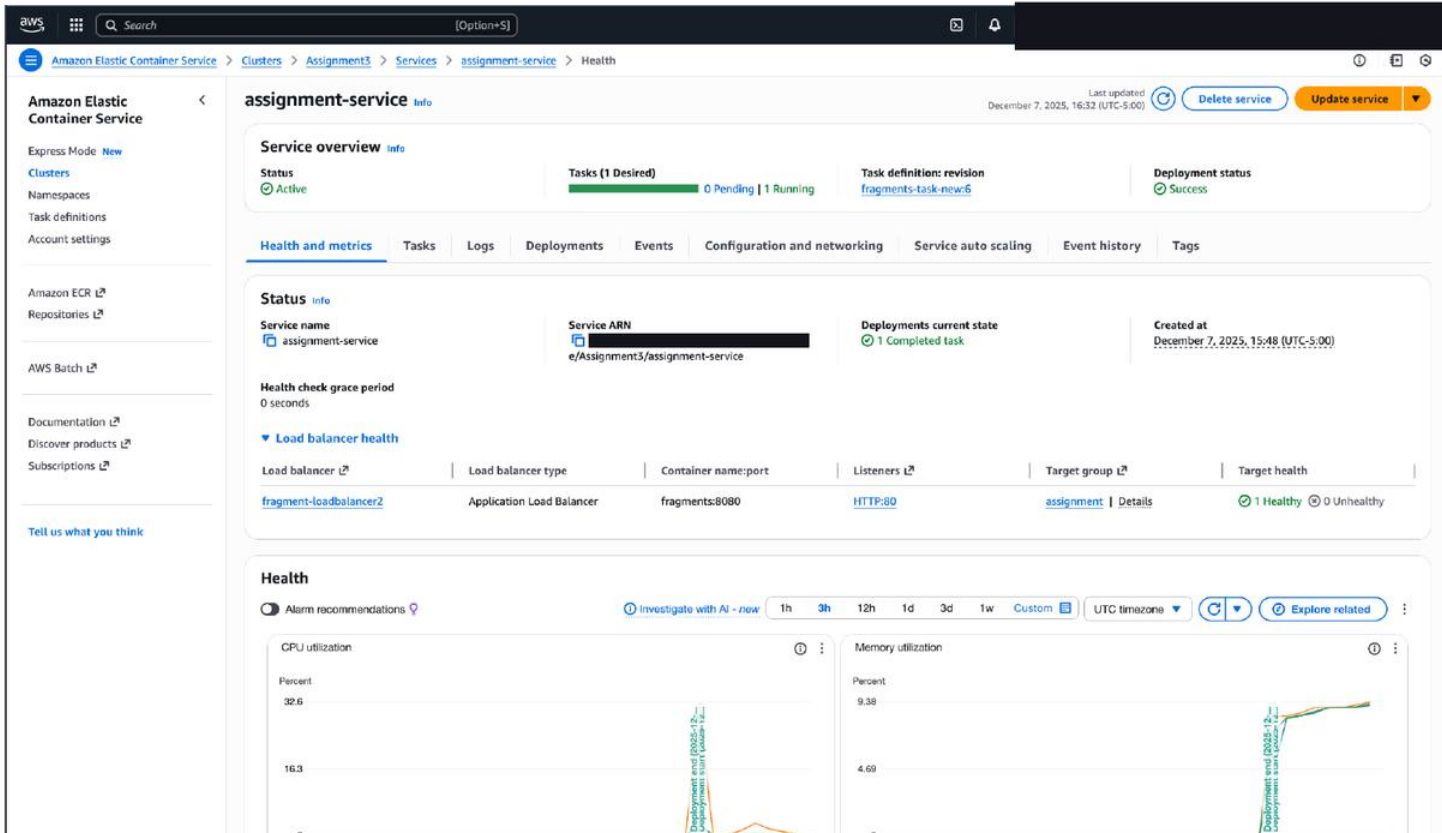
The AWS. Snapshot: Syed Nabeel Raza

The screenshot shows the Amazon Elastic Container Service (ECS) Services overview page for the Assignment3 cluster. The left sidebar is the same as the previous screenshot. The main content area displays the cluster overview for Assignment3, showing its ARN, status (Active), CloudWatch monitoring (Default), and registered container instances (1). Below the overview, there are tabs for Services, Tasks, Infrastructure, Metrics, Scheduled tasks, Configuration, Event history, and Tags. The Services tab is selected, showing a table of services. The table has columns for Service name, ARN, Status, Scheduling strategy, Launch type, Task definition, Deployments and tasks, and Last updated. The service listed is assignment-service.

Service name	ARN	Status	Scheduling strategy	Launch type	Task definition	Deployments and tasks	Last updated
assignment-service	arn:aws:ecs:us-east-1:123456789012:cluster/Assignment3/service/assignment-service	Active	REPLICA	FARGATE	fragments-task-new:6	1/1 Tasks running	December 7, 2025, 16:51 (UTC-5:00)

The AWS. Snapshot: Syed Nabeel Raza

ECS Service Details (assignment Service)



The AWS. Snapshot: Syed Nabeel Raza

Running Tasks (ECS Task Tabs)

The screenshot displays the Amazon Elastic Container Service (ECS) console for a specific task. The left sidebar shows the navigation menu with options like Express Mode, Clusters, Namespaces, Task definitions, Account settings, Amazon ECR, Repositories, AWS Batch, Documentation, Discover products, and Subscriptions. The main content area is titled with the task ID 67395fbca4be40069b6e3a7fba982a97. It includes tabs for Configuration, Metrics, Logs, Networking, Volumes (0), and Tags. The 'Task overview' section shows the task is in a 'Running' state, with details on its ARN, last status, desired status, and start/creation time. Below this, the 'Fargate ephemeral storage' section shows encryption and size (20 GiB). The 'Configuration' section provides details on the operating system (Linux/X86_64), CPU and memory (25 vCPU, 5 GB), platform version (1.4.0), task execution role, task role, fault injection, and ECS execution status. It also lists the capacity provider, launch type (Fargate), task definition revision, task group, ENI ID, network mode (awsvpc), subnet, public IP, and MAC address. At the bottom, there are tabs for container details for fragments, including Details, Log configuration, Restart policy, Network bindings, Docker labels and hosts, Environment variables and files, and Volume configuration.

The AWS. Snapshot: Syed Nabeel Raza

The screenshot shows the Amazon Elastic Container Service (ECS) console for task definitions. The breadcrumb navigation indicates the path: Container Service > Task definitions > fragments-task-new. The main heading is 'fragments-task-new (3)' with an 'info' link. A search bar and a filter for 'Active' status are present. A table lists three task definition revisions, all with an 'Active' status. The table has columns for 'Task definition: revision' and 'Status'. The revisions are 'fragments-task-new3', 'fragments-task-new2', and 'fragments-task-new1'. The console also shows a 'Deploy' button and an 'Actions' dropdown menu.

The AWS. Snapshot: Syed Nabeel Raza

Task Definition

The screenshot shows the AWS Management Console interface for the 'assignment-service' task definition. The left sidebar contains navigation links for Amazon Elastic Container Service, Clusters, Assignment3, Services, assignment-service, and Tasks. The main content area is titled 'assignment-service' and includes a 'Service overview' section with a status of 'Active' and a task definition of 'revision fragments-task-new:5'. Below this, there are tabs for 'Tasks', 'Logs', 'Deployments', 'Events', 'Configuration and networking', 'Service auto scaling', 'Event history', and 'Tags'. The 'Tasks' tab is selected, showing a table of tasks. The table has columns for Task, Last status, Desired st..., Tas..., Health sta..., Created at, Started by, Started at, and Container instan... The first task is '67395fbc4be40069b6e3...' with a status of 'Running' and a health status of 'Unknown'. Below the tasks table, there is a section for 'Containers for task 67395fbc4be40069b6e3a7fba982a97'. This section shows a table of containers with columns for Container name, Container runtime ID, Image URI, Image Digest, Status, Health status, CPU, and Memory hard/soft limit. The first container is 'fragments' with a status of 'Running' and a health status of 'Unknown'.

Service overview

Status: Active Tasks (1 Desired): 0 Pending | 1 Running Task definition: revision fragments-task-new:5 Deployment status: Success

Health and metrics **Tasks** Logs Deployments Events Configuration and networking Service auto scaling Event history Tags

Tasks (1/1)

Filter tasks by property or value Filter desired status: Any desired status Filter launch type: Any launch type

Task	Last status	Desired st...	Tas...	Health sta...	Created at	Started by	Started at	Container instan...
67395fbc4be40069b6e3...	Running	Running	fragm...	Unknown	43 minutes ago	ecs-svc/14108302158...	42 minutes ago	-

Containers for task 67395fbc4be40069b6e3a7fba982a97

Containers (1)

Filter containers

Container name	Container runtime ID	Image URI	Image Digest	Status	Health status	CPU	Memory hard/soft limit
fragments	67395fbc4be40...	sha256:ab...	sha256:ab...	Running	Unknown	0	-/-

The AWS. Snapshot: Syed Nabeel Raza

Load Balancer Overview (ALB)

The screenshot displays the AWS Management Console interface for an Application Load Balancer (ALB) named 'fragment-loadbalancer2'. The left sidebar shows the navigation menu with categories like Instances, Images, Elastic Block Store, Network & Security, Load Balancing, and Auto Scaling. The main content area is divided into two sections: 'Details' and 'Resource map'.

Details Section:

- Load balancer type:** Application
- Scheme:** Internet-facing
- Status:** Active
- Hosted zone:** Z35SXDOTRQ7X7K
- VPC:** vpc-01b5c63a15fba8650
- Availability Zones:** us-east-1c (use1-az1), us-east-1d (use1-az2), us-east-1e (use1-az3), us-east-1b (use1-az6), us-east-1f (use1-az5), us-east-1a (use1-az4)
- Load balancer IP address type:** IPv4
- Date created:** December 6, 2025, 03:41 (UTC-05:00)
- Load balancer ARN:** arn:aws:elasticloadbalancing:us-east-1:123456789012:loadbalancer/app/fragment-loadbalancer2:0c70c
- DNS name:** fragment-loadbalancer2-0c70c.elb.amazonaws.com

Resource map Section:

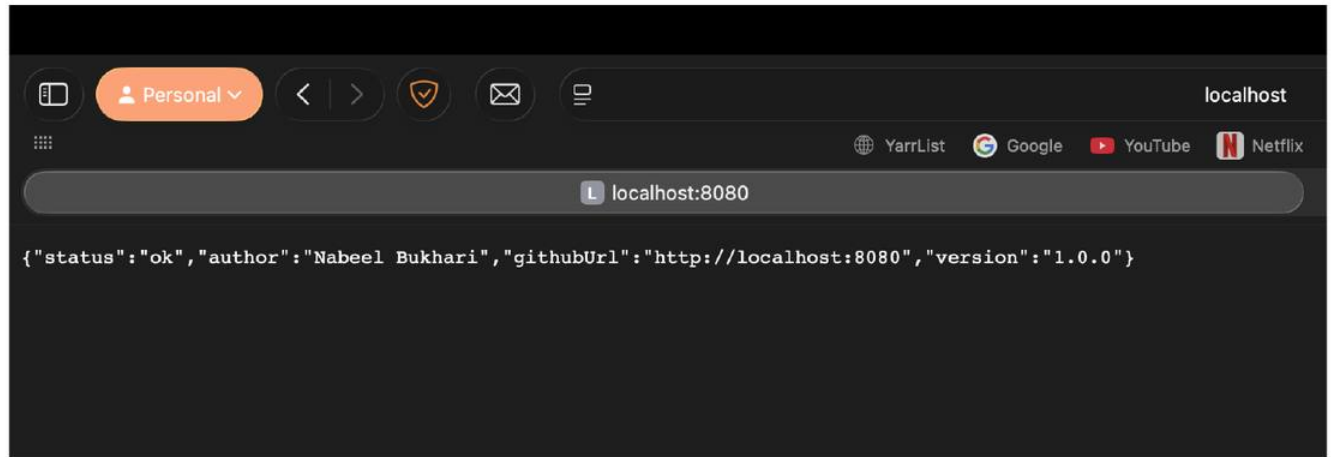
The 'Resource map' section provides a visual overview of the load balancer's architecture. It shows the following components and their connections:

- Listeners (1):** HTTP-80
- Rules (1):** Priority default, Forward to target group. Conditions (if): If no other rule applies.
- Target groups (1):** IP, HTTP assignment. 1 target.
- Targets (1):** 172.31.27.41, Port 8080. Status: Healthy.

The resource map also includes a 'Show resource details' toggle and a 'Give feedback' button.

The AWS. Snapshot: Syed Nabeel Raza

Health Check

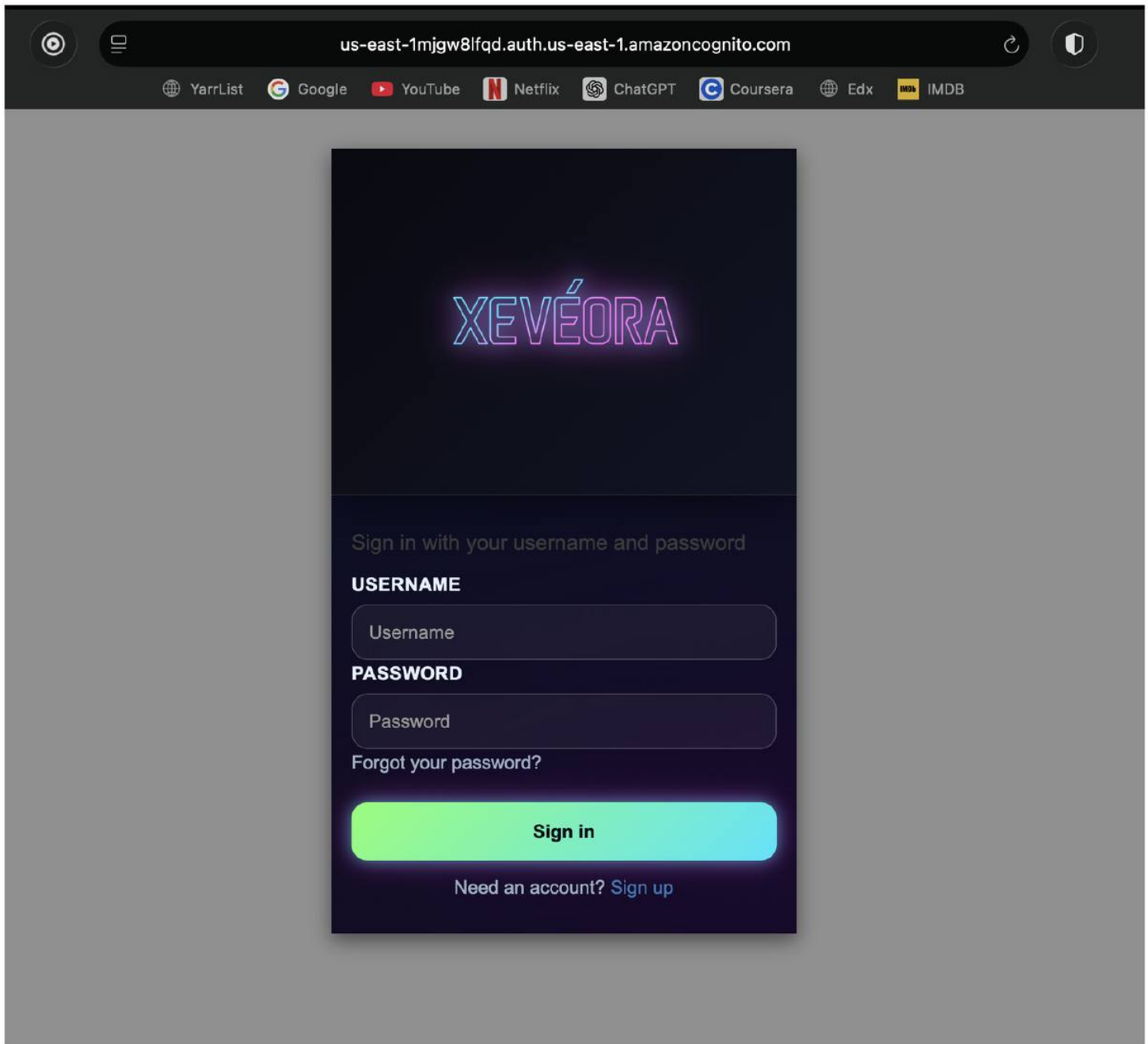


The Safari. Snapshot: Syed Nabeel Raza

UI

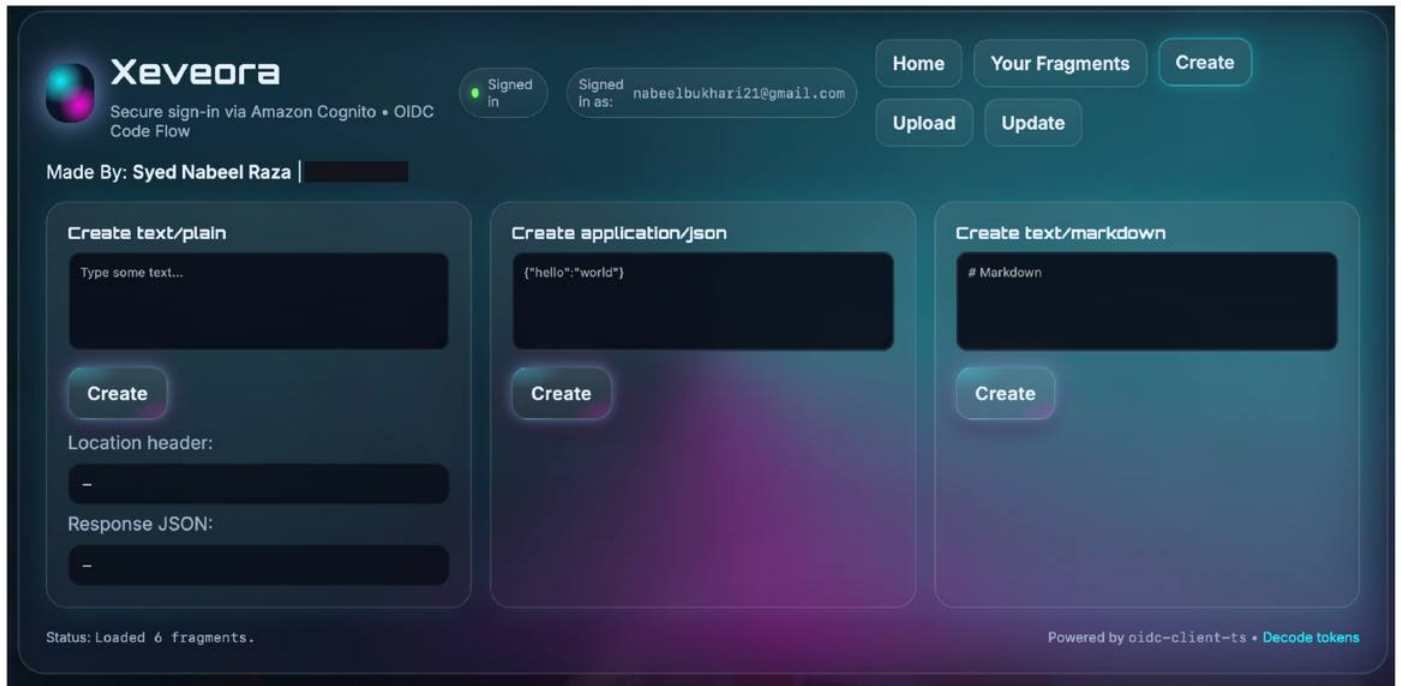
Screenshots

Cognito Hosted UI Login



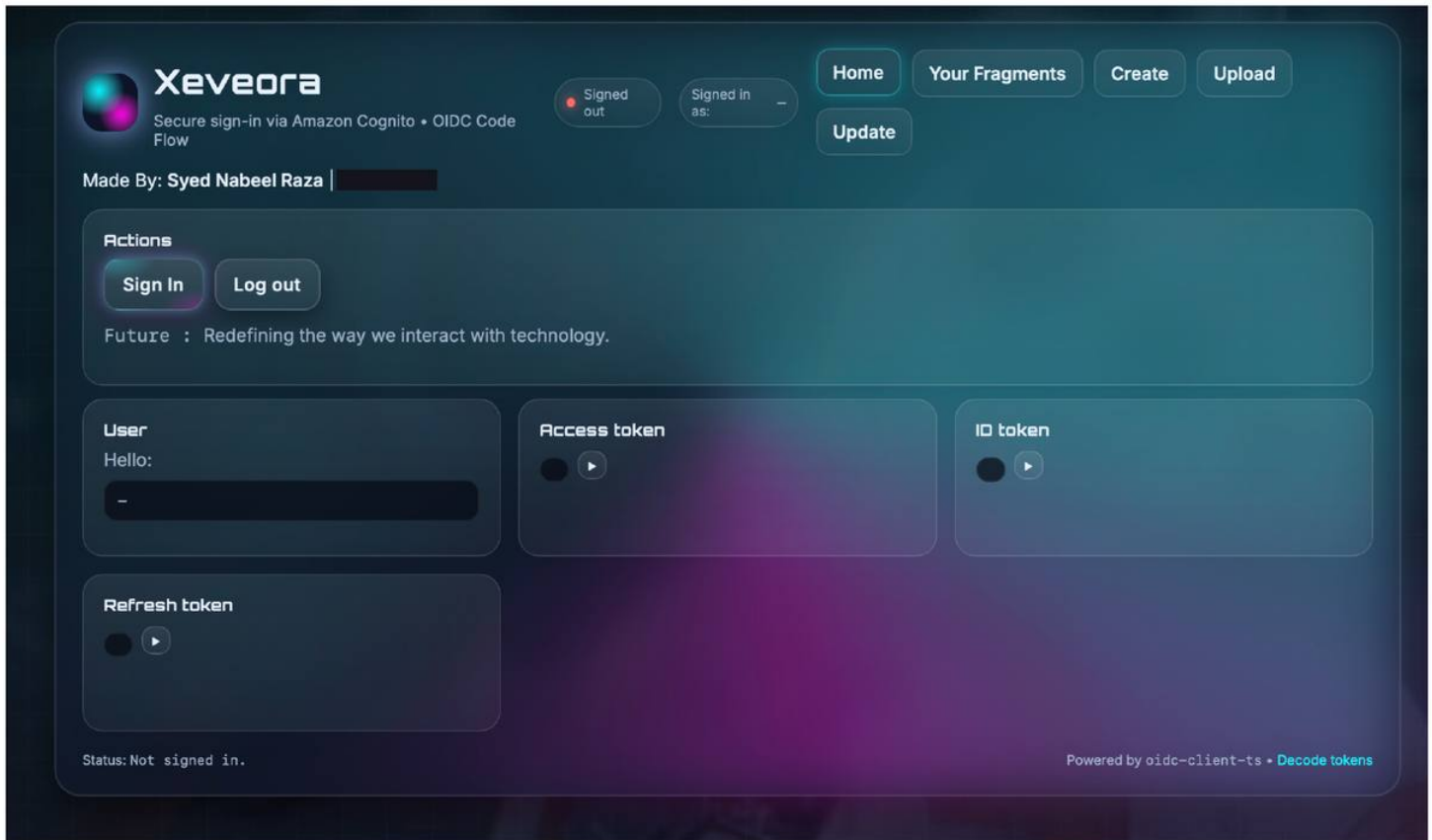
The Safari. Snapshot: Syed Nabeel Raza

UI After Login



The Safari. Snapshot: Syed Nabeel Raza

UI Before Login



The Safari. Snapshot: Syed Nabeel Raza

Fragments Page

**Xeveora**
Secure sign-in via Amazon Cognito • OIDC Code Flow

Signed in: nabeelbukhari21@gmail.com

[Home](#) [Your Fragments](#) [Create](#)

[Upload](#) [Update](#)

Made By: Syed Nabeel Raza |

Your fragments

Refresh

id=76680ccf-18d2-4701-ae84-6e5fd02482f4size=5 | 2025-12-07T21:29:35.032Z
text/plain; charset=utf-8
[View](#) [Edit](#) [Delete](#)

id=80eaa56b-adbd-4451-94cf-62ec535a3186size=4 | 2025-12-07T21:36:24.560Z
text/plain; charset=utf-8
[View](#) [Edit](#) [Delete](#)

id=9b1f1bd1-54a4-4847-87eb-ac3f2e394497size=9 | 2025-12-07T21:07:37.398Z
text/markdown; charset=utf-8
[View](#) [Edit](#) [Delete](#)

id=dd7266bb-0a32-40c9-ba40-b921e682b212size=3 | 2025-12-07T21:07:41.181Z
text/plain; charset=utf-8

Preview

Select a fragment to preview

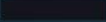
The Safari. Snapshot: Syed Nabeel Raza

Create Page

**Xeveora**
Secure sign-in via Amazon Cognito • OIDC Code Flow

Signed in
Signed in as: nabeelbukhari21@gmail.com

HomeYour FragmentsCreateUploadUpdate

Made By: Syed Nabeel Raza | 

Create text/plain

Type some text...

Create

Location header:
-

Response JSON:
-

Create application/json

```
{"hello": "world"}
```

Create

Create text/markdown

Markdown

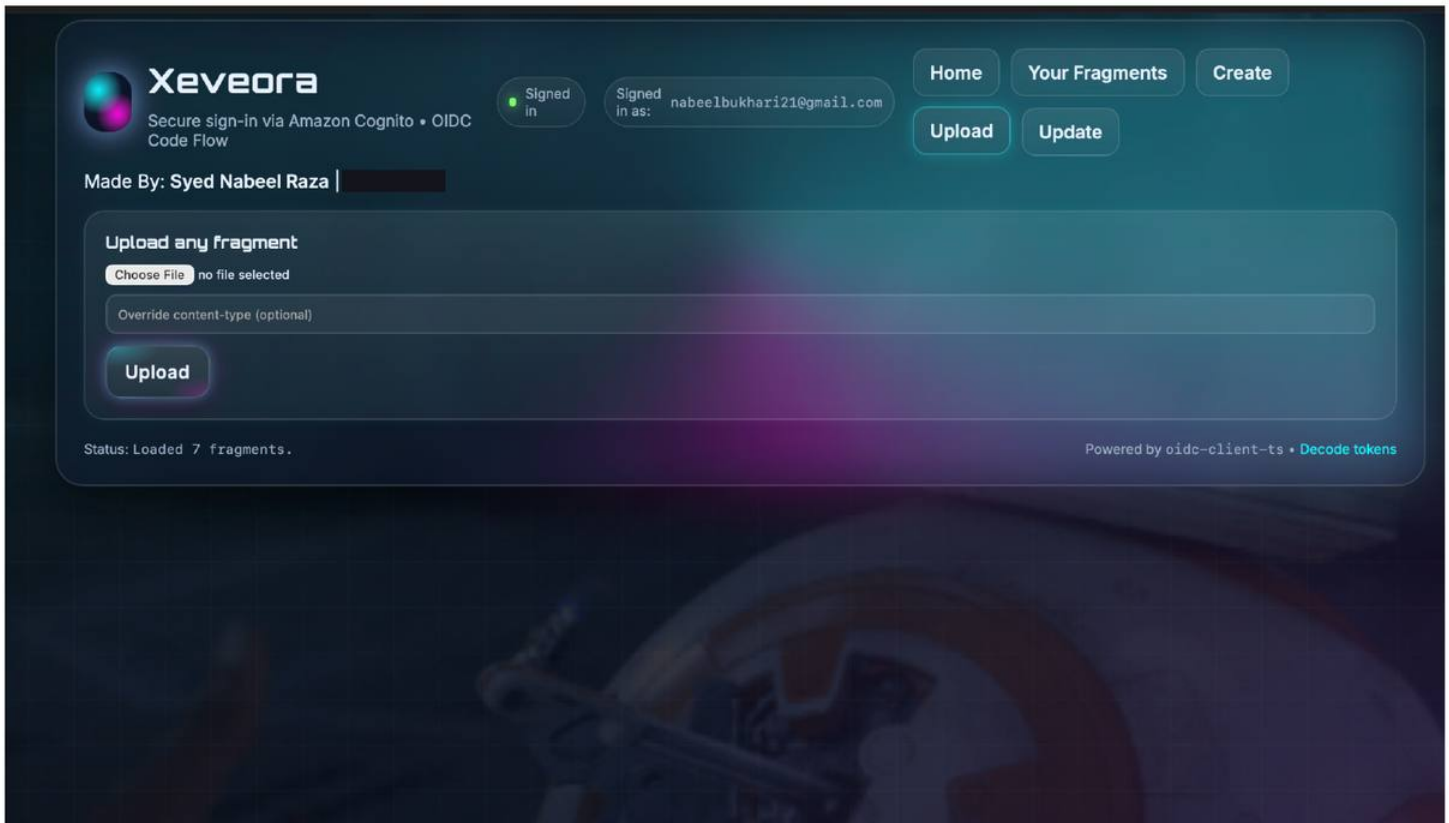
Create

Status: Loaded 6 fragments.

Powered by oidc-client-ts • [Decode tokens](#)

The Safari. Snapshot: Syed Nabeel Raza

Upload Page



The Safari. Snapshot: Syed Nabeel Raza

Preview Fragment

**Xeveora**
Secure sign-in via Amazon Cognito • OIDC Code Flow

Signed in
Signed in as: nabeelbukhari21@gmail.com

HomeYour FragmentsCreate

UploadUpdate

Made By: Syed Nabeel Raza

Your fragments

Refresh



id=76680ccf-18d2-4701-ae84-6e5fd02482f4size=5 | 2025-12-07T21:29:35.032Z

text/plain; charset=utf-8

ViewEditDelete



id=80eaa56b-adbd-4451-94cf-62ec535a3186size=4 | 2025-12-07T21:36:24.560Z

text/plain; charset=utf-8

ViewEditDelete



id=9b1f1bd1-54a4-4847-87eb-ac3f2e394497size=9 | 2025-12-07T21:07:37.398Z

text/markdown; charset=utf-8

ViewEditDelete



id=dd7266bb-0a32-40c9-ba40-b921e682b212size=3 | 2025-12-07T21:07:41.181Z

text/plain; charset=utf-8

Preview

text/plain; charset=utf-8#80eaa56b-adbd-4451-94cf-62ec535a3186

a

The Safari. Snapshot: Syed Nabeel Raza

Update Fragment

**Xeveora**
Secure sign-in via Amazon Cognito • OIDC
Code Flow

Signed In
Signed in as: nabeelbukhari21@gmail.com

HomeYour FragmentsCreate

UploadUpdate

Made By: Syed Nabeel Raza

Update fragment (PUT)

Fragment id

Content-Type e.g. text/plain

New content (text modes)

Or choose a file to replace

Choose File no file selected

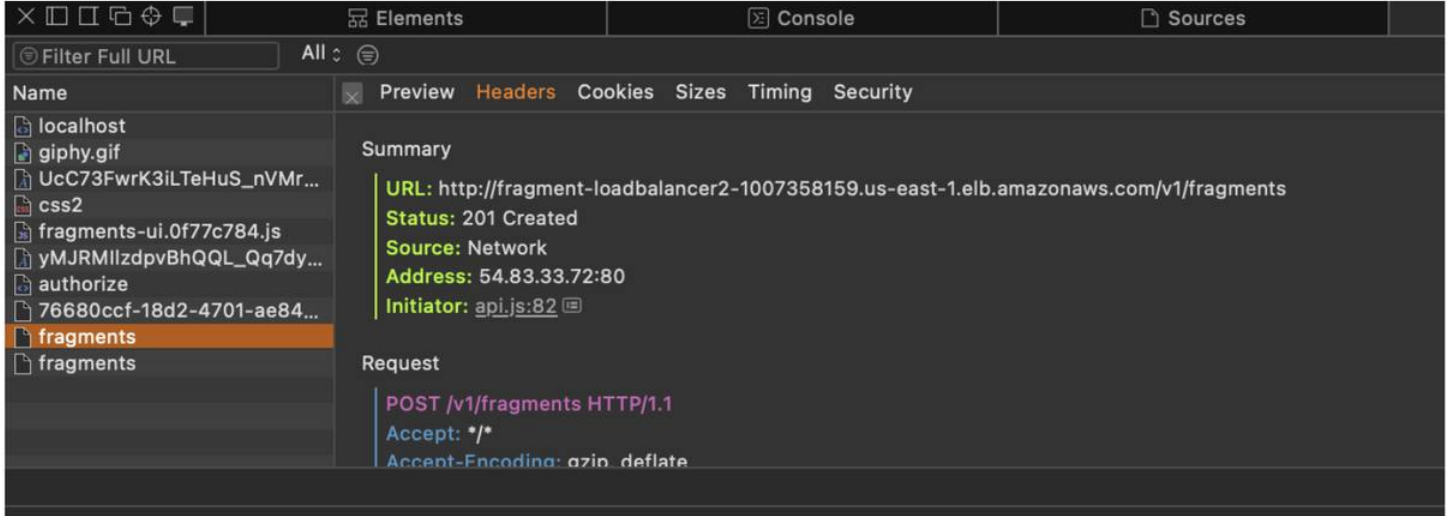
PUT /v1/fragments/:id

Status: Loaded 7 fragments.

Powered by oidc-client-ts • [Decode tokens](#)

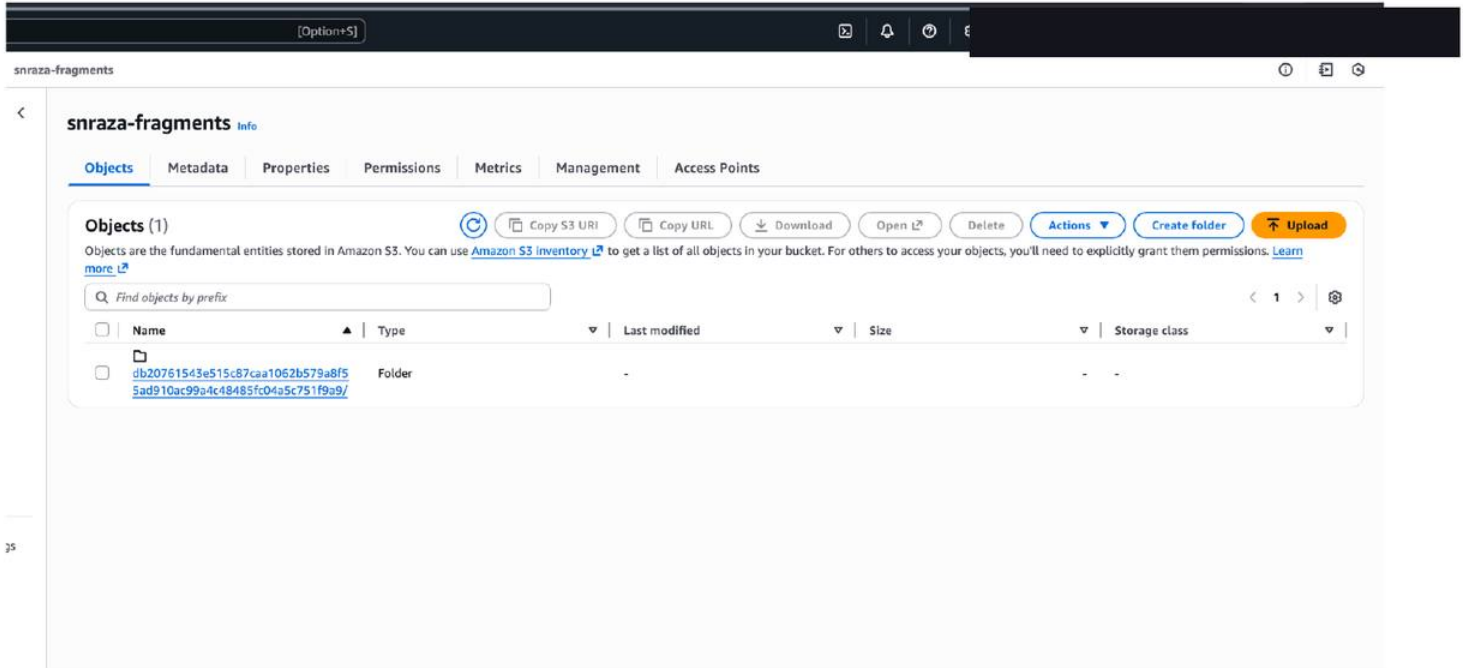
The Safari. Snapshot: Syed Nabeel Raza

All Success



The Safari. Snapshot: Syed Nabeel Raza

S3 Bucket



The AWS. Snapshot: Syed Nabeel Raza

ECR Repository

Amazon ECR

Private registry

Repositories

Images

Amazon Elastic Container Registry

▼ Private registry

Repositories

Images

Permissions

Lifecycle Policy

Repository tags

Features & Settings

▼ Public registry

Repositories

Settings

ECR public gallery

Amazon ECS

Amazon EKS

Getting started

Documentation

fragments

Summary

Images

Images (49)

Q Filter active images

🔄 Delete

📄 Copy URI

🔍 Details

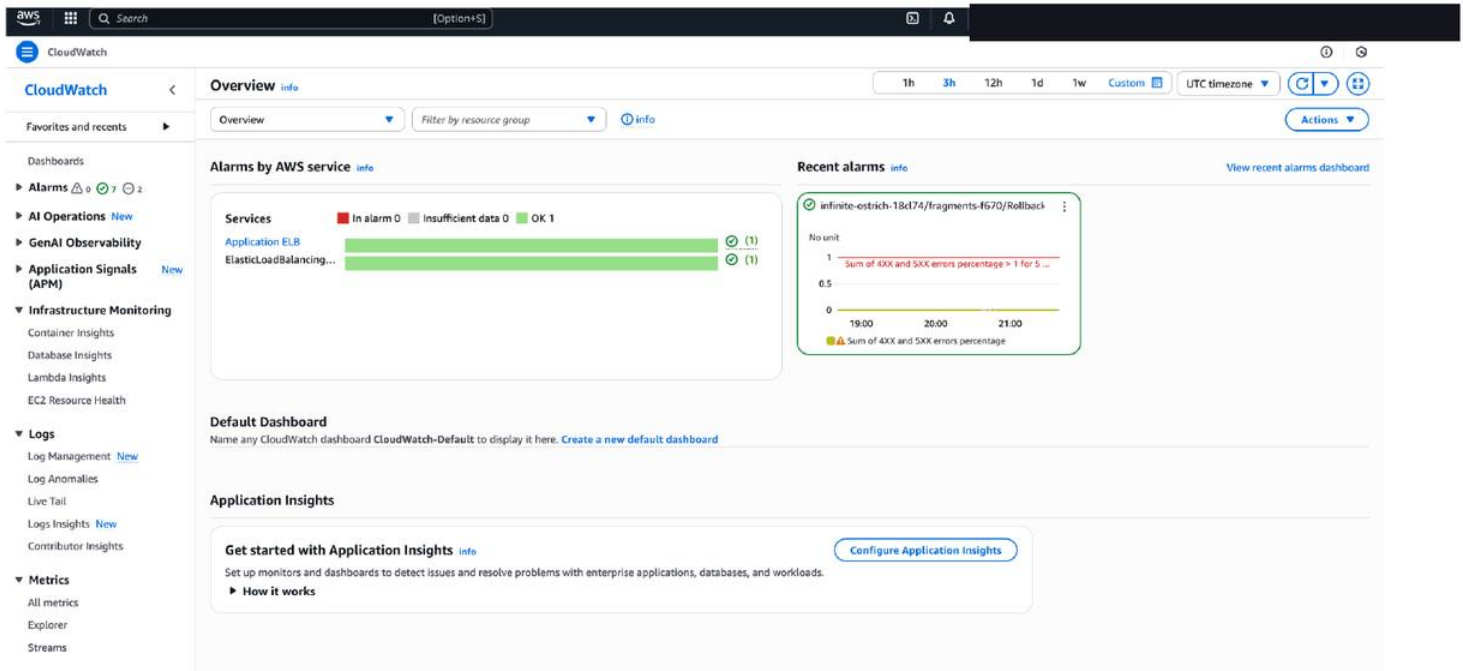
🔍 Scan

View push commands

☐	Image tags	Type	Created at	Image size	Image digest	Last pulled at
☐	v1.0.9	Image	December 07, 2025, 15:59:47 (UTC-05)	74.66	sha256:6a47f399c309dc...	-
☐	v1.0.8	Image	December 07, 2025, 15:55:15 (UTC-05)	74.66	sha256:ada6d897b6f35a...	-
☐	v1.0.5	Image	December 07, 2025, 15:19:10 (UTC-05)	74.66	sha256:6af5a4e48fe4665...	December 07, 2025, 15:37:45 (UTC-05)
☐	v1.0.4	Image	December 07, 2025, 15:11:24 (UTC-05)	74.66	sha256:cd346de9f60e65...	-
☐	v1.0.3	Image	December 07, 2025, 14:56:31 (UTC-05)	74.66	sha256:f96fec772ce07c...	-
☐	v1.0.2	Image	December 07, 2025, 14:55:02 (UTC-05)	74.66	sha256:e207cb393085e6...	-
☐	v0.10.3	Image Index	December 07, 2025, 12:36:28 (UTC-05)	59.49	sha256:ab3753a66fd5ed...	December 07, 2025, 15:49:39 (UTC-05)
☐	-	Image	December 07, 2025, 12:36:27 (UTC-05)	0.01	sha256:e31305d10336e9...	-
☐	-	Image	December 07, 2025, 12:36:27 (UTC-05)	59.49	sha256:93b4dfe559b37c...	December 07, 2025, 15:49:39 (UTC-05)
☐	v0.10.1	Image Index	December 07, 2025, 12:33:55 (UTC-05)	58.70	sha256:ae9fb7767f5210...	-
☐	-	Image	December 07, 2025, 12:33:55 (UTC-05)	0.01	sha256:1d575bba3a43b7...	-

The AWS. Snapshot: Syed Nabeel Raza

CloudWatch Logs



The AWS. Snapshot: Syed Nabeel Raza

IAM Roles

Identity and Access Management (IAM)

Search IAM

Roles (25) Info

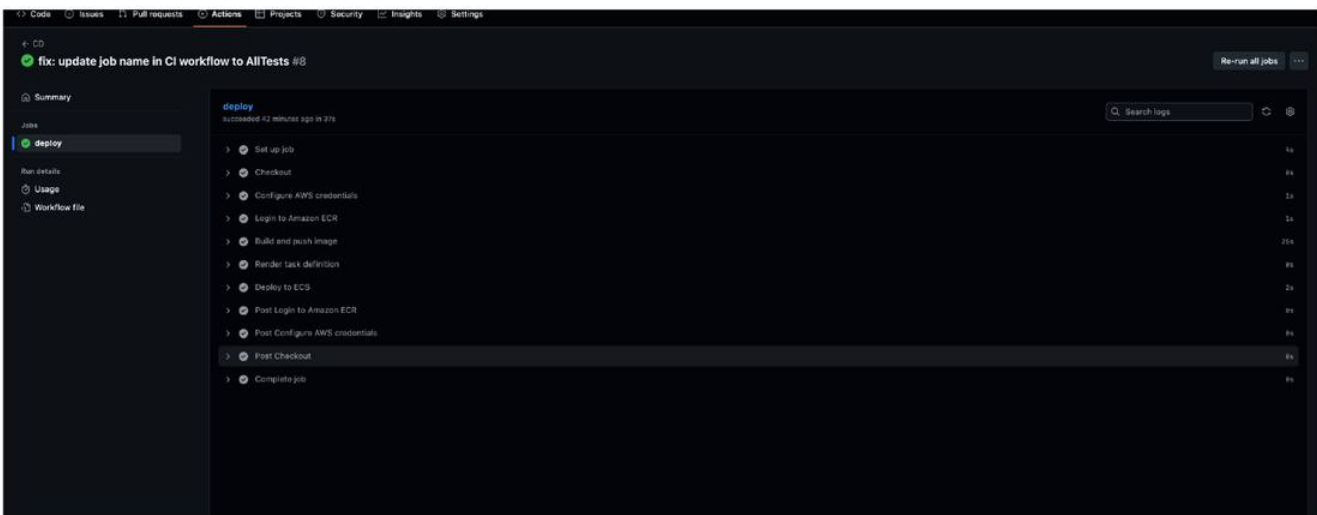
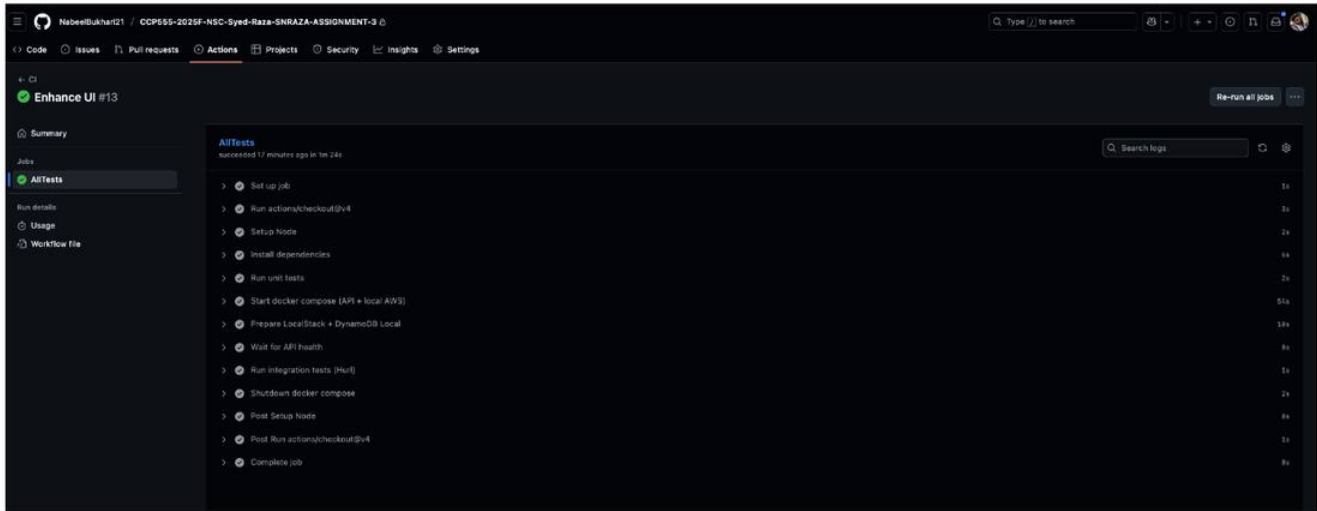
An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Search

☐	Role name	Trusted entities	Last activity
☐	AWSServiceRoleForApplicationAutoScaling_ECSService	AWS Service: ecs.application-autosco	35 minutes ago
☐	AWSServiceRoleForAutoScaling	AWS Service: autoscaling (Service-Li	7 minutes ago
☐	AWSServiceRoleForAWSCloud9	AWS Service: cloud9 (Service-Linked	-
☐	AWSServiceRoleForCloudWatchEvents	AWS Service: events (Service-Linked	-
☐	AWSServiceRoleForECS	AWS Service: ecs (Service-Linked Rol	5 minutes ago
☐	AWSServiceRoleForElasticCache	AWS Service: elasticsearch (Service-Lir	-
☐	AWSServiceRoleForElasticLoadBalancing	AWS Service: elasticloadbalancing (\$	51 minutes ago
☐	AWSServiceRoleForNATGateway	AWS Service: ec2-nat-gateway (Servi	48 minutes ago
☐	AWSServiceRoleForOrganizations	AWS Service: organizations (Service-	-
☐	AWSServiceRoleForSupport	AWS Service: support (Service-Linker	-
☐	AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor (Service	-
☐	[REDACTED]	AWS Service: eks	-
☐	c177553a4587946f [REDACTED]	AWS Service: ec2	-
☐	c177553a4587946f [REDACTED]	AWS Service: lambda	89 days ago
☐	EMR_AutoScaling_DefaultRole	AWS Service: application-autoscaling	-
☐	EMR_DefaultRole	AWS Service: elasticmapreduce	-
☐	EMR_EC2_DefaultRole	AWS Service: ec2	6 minutes ago
☐	EMR_Notebooks_DefaultRole	AWS Service: elasticmapreduce	-
☐	LabRole	Account: [REDACTED]	6 minutes ago

The AWS. Snapshot: Syed Nabeel Raza

CI/CD



The VS Code. Snapshot: Syed Nabeel Raza

Test Integration

```
● admin@Syeds-Macbook-Pro fragments % npm run test:integration

> fragments@1.0.0 test:integration
> hurl --test --glob "tests/integration/**/*.hurl"

Success tests/integration/health-check.hurl (1 request(s) in 118 ms)
Success tests/integration/post-fragments-unauthenticated.hurl (1 request(s) in 131 ms)
Success tests/integration/404.hurl (1 request(s) in 138 ms)
Success tests/integration/post-fragments-unsupported-type.hurl (1 request(s) in 139 ms)
Success tests/integration/post-fragments-json.hurl (2 request(s) in 468 ms)
Success tests/integration/post-fragments-charset.hurl (2 request(s) in 469 ms)
Success tests/integration/post-fragments.hurl (2 request(s) in 472 ms)
Success tests/integration/put-fragments.hurl (4 request(s) in 506 ms)
Success tests/integration/lab-9-s3.hurl (4 request(s) in 506 ms)
Success tests/integration/lab-10-dynamodb.hurl (8 request(s) in 537 ms)

-----
Executed files:      10
Executed requests: 26 (48.1/s)
Succeeded files:    10 (100.0%)
Failed files:        0 (0.0%)
Duration:            540 ms (0h:0m:0s:540ms)

admin@Syeds-Macbook-Pro fragments %
```

The VS Code. Snapshot: Syed Nabeel Raza

Test Coverage

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	87.27	70.34	94.11	87.82	
src	94.28	54.54	100	97.05	
app.js	96	50	100	100	40-42
logger.js	83.33	50	100	83.33	6
response.js	100	100	100	100	
src/model	98.11	69.23	100	100	
fragment.js	98.11	69.23	100	100	24,32,65-126
src/model/data	100	50	100	100	
index.js	100	50	100	100	5
src/model/data/memory	94.23	76	87.5	97.61	
index.js	100	100	100	100	
memory-db.js	94.11	76	87.5	97.56	48
src/routes	100	75	100	100	
index.js	100	75	100	100	7
src/routes/api	77.19	57.69	87.5	77.19	
fragments.js	76.14	57.69	87.5	76.14	...77,177-178,199-201,211-215
index.js	100	100	100	100	
src/utils	84.12	78.43	100	83.05	
convert.js	84.12	78.43	100	83.05	69,78,89,108,115-117,124-126
Test Suites: 7 passed, 7 total					
Tests: 22 passed, 22 total					
Snapshots: 0 total					
Time: 0.781 s, estimated 1 s					

The VS Code. Snapshot: Syed Nabeel Raza